

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Arquitectura de Computadores y Ensambladores 1

Vacaciones Primer Semestre

Modulo 1 – Media Aritmética Pondera
Invernadero Inteligente IoT
Proyecto 1

Grupo 19

Jennifer Michelle Rosales Juárez – 202400063

14 de junio de 2026

Módulo 1 – Media Ponderada

Nombre del Archivo: modulo_1_media.s

Nombre del Archivo de Salida: resultado_media.txt

1. Descripción

Este módulo calcula la media aritmética ponderada de una columna seleccionada del archivo lecturas.csv, usando los 30 registros disponibles y pesos crecientes del 1 a 30 (donde la fila 1 tiene un peso de 1 y la fila 30 tiene un peso de 30). La columna a procesar se indica mediante un argumento al ejecutar el programa. El resultado se escribe en *resultado_media.txt*. el algoritmo se ejecuta en las siguientes fases:

1. **Decodificación de Argumentos:** El programa lee el argumento de la línea de comandos desde el stack pointer (sp) para aislar el número de la columna que se desea analizar. El valor es convertido de su representación ASCII a un entero puro restandole el offset #48.
2. **Carga de Datos Orientada al Stack:** Se invoca la subrutina externa *utils_read_column_to_stack*, la cual realiza el análisis sintáctico del archivo csv y almacena consecutivamente en la pila del sistema los 30 valores correspondientes a la columna solicitada.
3. **Procesamiento Aritmético en Ciclo:** se recorren los datos mapeados en el stack aplicando de forma iterativa la ecuación de acumulación:
 - **Suma simple** ($\sum X_i$) = acumulación directa de cada dato extraído
 - **Suma ponderada** ($\sum (X_i * W_i)$) = cada dato se multiplica de forma lineal por su peso dinámico correspondiente (W_i), el cual inicia en 1 y se incrementa uno a uno en cada iteración hasta alcanzar el límite físico de 30 datos.
4. **Calculo de la Media Ponderada:** Al finalizar las 30 iteraciones, se divide el acumulador ponderado ($\sum (X_i * W_i)$) entre la sumatoria constante de los pesos (465, resultado matemático exacto de la suma de enteros del 1 al 30). La división se procesa mediante la instrucción *udiv*.
5. **Formateo de Buffer y Persistencia:** Los resultados numéricos se transforman a cadena de caracteres ASCII utilizando la función *utils_itoa*. Se construye secuencialmente un reporte estructurado en memoria limpia (*out_buffer*) mediante subrutinas de copia de cadenas byte por byte y, finalmente, se escribe físicamente a disco invocado a *utils_write_result*.

2. Explicación de los Registros Utilizados

Para garantizar el cumplimiento de la convención de llamadas de ARM64, se dividieron los registros de la CPU en unidades de control, almacenamiento y manipulación aritmética.

x0	Registro de Argumentos / Dirección Fuente	Utilizado inicialmente para capturar la dirección del argumento de consola. Posteriormente actúa como el registro de paso de parámetros hacia las funciones del archivo <i>utils.s</i> (recibe el número a convertir para <i>utils_itoa</i> y la dirección del buffer de escritura)
x1	Longitud/ Buffer de Destino	Almacena las longitudes de los strings fijos y define la dirección del buffer intermedio (<i>num_buffer</i>) durante las conversaciones numéricas.
w11 /x11	Registro de Trabajo y Control Crítico	Recibe el identificador numérico de la columna. Al final del procesamiento aloja el resultado de la división de la media ponderada antes de sufrir la sobreescritura
x20	Resguardo del Stack Pointer Original	Almacena de manera segura la dirección inicial de <i>sp</i> recibida por el sistema operativo antes de las alteraciones por lectura de datos.
x24	Puntero Base Inferior	Almacena de forma inalterable el límite inferior (dirección más baja) de los 30 datos numéricos cargados en la pila.
x25	Puntero Límite Superior	Contiene la dirección de memoria más alta de la pila donde se encuentra el primer dato elegible del ciclo
x26	Resguardo de Limpieza del Stack	Copia la dirección del puntero de pila para su posterior restauración manual una vez concluido el ciclo aritmético
x4	Acumulador de Suma Simple	Registro de persistencia dedicado a calcular secuencialmente la sumatoria de los datos
x5	Acumulador de Suma Ponderada	Registro dedicado a almacenar el resultado parcial y final de la acumulación del producto <i>dato*peso</i>
x14	Contador de Pesos	Variables de control de ciclo que actúa como el peso lineal multiplicativo; inicializado en 1 y autoincrementando de manera entera hasta 30

x7	Puntero de Lectura Dinámica	Registro que decrece en pasos constantes de 16 bytes (<i>sub x7, x7, #16</i>) para indexar y extraer consecutivamente cada dato guardado en la pila
x8	Registro de carga inmediata	Aloja temporalmente el dato de 64 bits extraído de la memoria RAM (<i>ldr x8, [x7]</i>) para operar las adiciones y multiplicaciones
x9	Multiplicación Intermedia	Guarda el producto transitorio de la instrucción <i>mul x9, x8, x14</i> antes de sumarse a x5
x10	Denominador Constante	Registro de carga inmediata configurado con el valor entero 465 para efectuar la división
x23	Registro de Resguardo de Seguridad del Modulo	Registro no volátil de alta prioridad implementado para salvar el resultado de la media ponderada (<i>mov x23, x11</i>) antes de invocar subrutinas externas, evitando la perdida de información debida al bug detectado
x15	Cursor Dinámico de Escritura	Puntero que controla la posición exacta de memoria dentro de <i>out_buffer</i> donde se concatenan las cadenas formateadas.
x29	Frame Pointer	Control de flujo de hardware para el manejo de marcos de activación
x30	Link Register	Control de almacenamiento de la dirección de retorno de subrutinas

3. Explicación de Ciclos, Salto y Subrutinas

Ciclo de Carga y Procesamiento (*ciclo_suma*)

El bucle implementa un enfoque de recorrido inverso basada en punteros de memoria de alta velocidad

```
ciclo_suma:
    sub x7, x7, #16           // Decrementa el puntero en 2 palabras (16 bytes)
    cmp x7, x24               // Evalúa si se alcanzó el límite inferior del Stack
    blt fin_ciclo             // Salto condicional: si x7 < x24, finaliza la iteración
```

Mecanismo de Salto: se utiliza un salto condicional *blt* (Branch if Less Than) posterior a una instrucción de comparación *cmp*. Al procesarse de forma descendente, en el momento en el que el puntero dinámico x7 sobrepasa el limite inferior registrado en

x24, el ciclo rompe su flujo de ejecución y desvía a la CPU hacia la etiqueta *fin_ciclo*. En caso contrario, ejecuta las instrucciones aritméticas y realiza un salto incondicional *b ciclo_suma* hacia la cabecera.

Subrutina *copiar_string*

Función interna diseñada para la transferencia en bloque de cadena estáticas con longitudes conocidas.

Control de Flujo: Evalua el contador de bytes (x17) mediante la instrucción de bifurcación condicional sobre cero *cbz x17, copiar_fin*. Mientras queden bytes pendientes, ejecuta una carga con post-incremento (*ldrb w18, [x16], #1*) y almacenamiento con post-incremento (*strb w18, [x15], #1*), desplazando el cursor dinámico x15 automáticamente hacia adelante

Subrutina *copiar_cstr*

Subrutina de acoplamiento para strings de longitud variables generados dinámicamente en tiempo de ejecución (como lo numero procesados *utils_itoa*).

- Mecanismo de Parada: Lee byte por byte la dirección fuente de forma secuencial. Implementa una doble condición de salida: finaliza inmediatamente si encuentra un carácter nulo (*cbz w16, cstr_fin*) o si detecta un salto de línea (*cmp x16, #10 seguido de beq cstr_fin*) protegiendo el buffer de desbordamiento en memoria.

4. Explicación del Formato de entrada y Salida

Formato de entrada

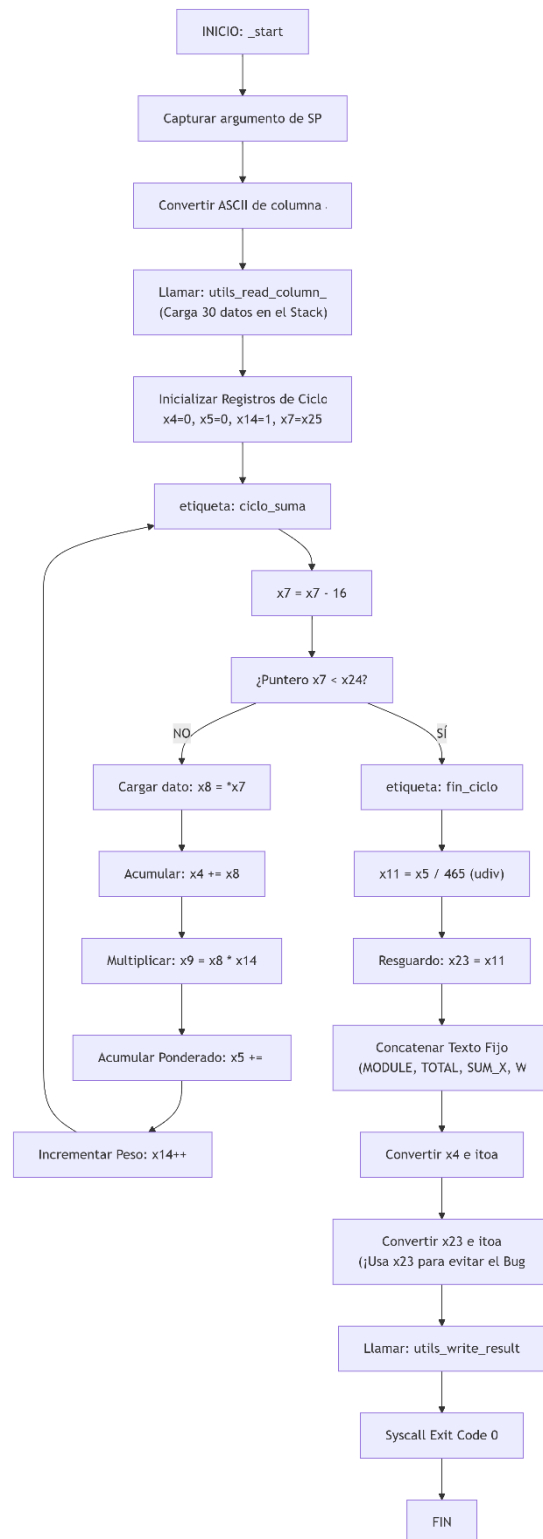
El modulo procesa datos estructurados de un archivo en formato de valores separados por comas (.csv), compuesto por filas de registros de texto numérico. El programa requiere como parámetro de entrada en consola un carácter ASCII en el rango de 1 a 9, el cual define la columna indexada que será analizada. Los datos de texto son parseados a binarios puros de 64 bits por la subrutina externa de lectura antes de ser inyectados en el ciclo de calculo.

Formato de Salida

El resultado final es exportado en texto plano dentro del archivo *resultado_media.txt*. La salida respeta estrictamente un formato de llaves e identificadores estructurados mediante saltos de línea (\n).

```
MODULE=WEIGHTED_MEAN
TOTAL_VALUES=30
SUM_X=[Valor calculado y transformado a ASCII]
WEIGHT_SUM=465
WEIGHTED_MEAN=[Valor final de la operación udiv transformado a ASCII]
```

5. Flujo del Modulo



6. Depuración con GDB

Análisis Técnico del Bug en la Subrutina utils_itoa

Durante la fase de integración del módulo, se detectó un comportamiento anómalo (bug de librería) al invocar la función externa utils_itoa. La convención de llamadas de ARM64 establece que los registros x0 al x7 son volátiles (pueden ser modificados por funciones externas), mientras que del x19 al x28 son no-volátiles (deben preservarse).

El resultado de la división de la Media Ponderada se almacena inicialmente en el registro temporal x11. Al invocar la subrutina utils_itoa para convertir la suma simple (SUM_X), la subrutina externa corrompe el registro x11 internamente (dejándolo con un valor de 10 debido al manejo de cadenas o saltos de línea). Para solucionar este problema de raíz y evitar que el reporte final imprimiera basura, se implementó un mecanismo de resguardo seguro: inmediatamente después de la división, el valor correcto se copia mediante `mov x23, x11` hacia el registro no-volátil x23. Al momento de escribir la Media Ponderada en el buffer, se recupera el valor desde `mov x0, x23`, garantizando la persistencia e integridad de los datos calculados.

Compilar y Enlazar

```
micelin000@LAPTOP-NAKKVTSI:~/modulo$ aarch64-linux-gnu-as -o utils.o utils.s
micelin000@LAPTOP-NAKKVTSI:~/modulo$ aarch64-linux-gnu-as -o modulo_1_media.o modulo_1_media.s
micelin000@LAPTOP-NAKKVTSI:~/modulo$ aarch64-linux-gnu-ld -o modulo_1_media utils.o modulo_1_media.o
```

Verificar que .csv termina con \$

```
micelin000@LAPTOP-NAKKVTSI:~/modulo$ tail -c 5 lecturas.csv
,0
$
```

Ejecutar normal para confirmar que funciona

```
micelin000@LAPTOP-NAKKVTSI:~/modulo$ qemu-aarch64 ./modulo_1_media 2
micelin000@LAPTOP-NAKKVTSI:~/modulo$ echo $?
0
micelin000@LAPTOP-NAKKVTSI:~/modulo$ cat resultado_media.txt
MODULE=WEIGHTED_MEAN
TOTAL_VALUES=30
SUM_X=847
WEIGHT_SUM=465
WEIGHTED_MEAN=28
```


EVIDENCIA – Depuración con GBD

Terminal 1 – arranca el programa en modo servidor

```
micelin000@LAPTOP-NAKKVTSI:~/modulo$ qemu-aarch64 -g 1234 ./modulo_1_media 2
```

Terminal 2 – conecta con GBD

```
micelin000@LAPTOP-NAKKVTSI:~$ gdb-multiarch -q modulo_1_media
modulo_1_media: No such file or directory.
```

```
(gdb) target remote :1234
Remote debugging using :1234
Reading /home/micelin000/modulo/modulo_1_media from remote target...
Warning: File transfers from remote targets can be slow. Use "set sysroot" to access files locally instead.
Reading /home/micelin000/modulo/modulo_1_media from remote target...
Reading symbols from target:/home/micelin000/modulo/modulo_1_media...
(No debugging symbols found in target:/home/micelin000/modulo/modulo_1_media)
warning: BFD: warning: system-supplied DSO at 0x400000810000 has a corrupt string table index
0x000000000400368 in _start ()
(gdb) break ciclo_suma
Breakpoint 1 at 0x4003a4
(gdb) break fin_ciclo
Breakpoint 2 at 0x4003c8
(gdb) continue
Continuing.

Breakpoint 1, 0x0000000004003a4 in ciclo_suma ()
```

Inicialización del entorno de depuración en GBD, configuración de puntos de parada (breakpoints) y detención en la primera iteración de ciclo_suma

```
(gdb) info registers x4 x5 x14 x24 x25 x7
x4             0x0             0
x5             0x0             0
x14            0x1             1
x24            0x4000007ffb00    70368752564992
x25            0x4000007ffce0    70368752565472
x7             0x4000007ffcd0    70368752565456
```

Inspección del estado inicial del banco de registros de control y determinación de los límites de direccionamiento del Stack Pointer previo de la acumulación

- **X4** = SUM_X
- **X5** = WEIGHTED_SUM
- **X14** = W_i, peso
- **X24 y X25** = límites del stack donde están los 30 datos leídos del csv

```
(gdb) delete 1
(gdb) continue
Continuing.

Breakpoint 2, 0x00000000004003c8 in fin_ciclo ()
(gdb) continue
```

Ruptura controlada de la iteración mediante la eliminación del breackpoint activo, avance del program counter directo hacia la etiqueta fin_ciclo

Ejecución secuencial de instrucción de maquina para el procesamiento de la división entera de hardware y validación de la constante WEIGHTED_MEAN

```
(gdb) print/d $x4
$1 = 847
(gdb) print/d $x5
$2 = 13056
```

```
(gdb) stepi 2
0x00000000004003d0 in fin_ciclo ()
(gdb) print/d $x11
$3 = 28
```

```
(gdb) continue
Continuing.
[Inferior 1 (process 701) exited normally]
(gdb) quit
```

Salida